A survey on LLM Agents: Architecture, Applications, and Challenges

Donghun Lee
Computer Science and Engineering
Chung-Ang University
Seoul, South Korea
ldh019@cau.ac.kr

Hyosu Kim
Computer Science and Engineering
Chung-Ang University
Seoul, South Korea
hskimhello@cau.ac.kr

2026 40th International Conference on Information Networking (ICOIN) | 979-8-3315-7896-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICOIN68469.2026.11480623

***Abstract*—Large Language Models (LLMs) have revolutionized natural language processing, but they lack autonomous interaction capabilities. LLM agents address this by integrating three key modules, such as memory, planning, and tool use modules, around the LLM core. This survey systematically analyzes LLM agents through their architectural components, control flow paradigms, and application domains spanning data analysis, software engineering, and social simulation. We identify critical challenges in reliability, evaluation, and scalability, and outline future research directions. This work provides a structured overview of the rapidly evolving landscape of LLM agents.**

***Index Terms*—LLM Agents, Agent Architecture, Multi-Agent Systems, Autonomous AI**

I. Introduction

The recent advancements in Large Language Models (LLMs) such as GPT-4 and Llama 2 have revolutionized the field of Natural Language Processing, demonstrating exceptional capabilities in language understanding, reasoning, and generation. However, these LLMs inherently possess knowledge confined to their training data and face limitations in autonomously interacting with external environments or formulating multi-step plans to achieve complex, long-term goals.

To overcome this limited autonomy of LLMs and extend their intelligence into actual executable behavior, the paradigm of LLM agents has rapidly emerged. An LLM agent utilizes the LLM as its brain, integrating modules such as memory, planning, and tool use to enable autonomous decision-making and action. These agents are demonstrating remarkable potential across a wide range of applications, from simple question answering to software development, data analysis, and complex environment simulation.

The field has seen rapid growth since 2023, with diverse agent architectures emerging for various applications [1]–[5]. This survey provides a systematic analysis of the architectural mechanisms and goal-oriented applications of LLM agents.

This survey aims to provide a systematic overview of LLM agents by focusing on key analytical dimensions. First, we provide a structural analysis by categorizing core design elements and the resulting control flow paradigms that implement agent autonomy in Section II. Second, Section III present a classification of application domains, showcasing representative case studies. Finally, Section IV discusses current limitations, such as reliability and evaluation difficulties, and propose critical avenues for future directions in the field.

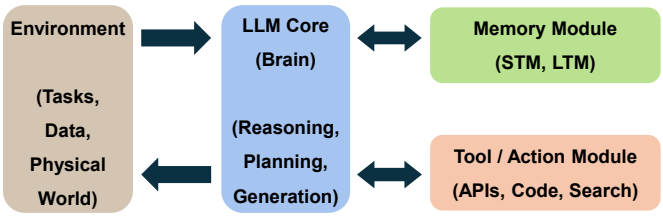


Fig. 1: **Overview of LLM agent architecture.** The system comprises three core components: the LLM Core for reasoning, planning and generation, the Memory Module for maintaining context (STM) and retrieving past experiences (LTM), and the Tool/Action Module for external interaction. Arrows indicate the agent-environment interaction cycle.

II. Structural Analysis: Core Design Elements and Control Flow

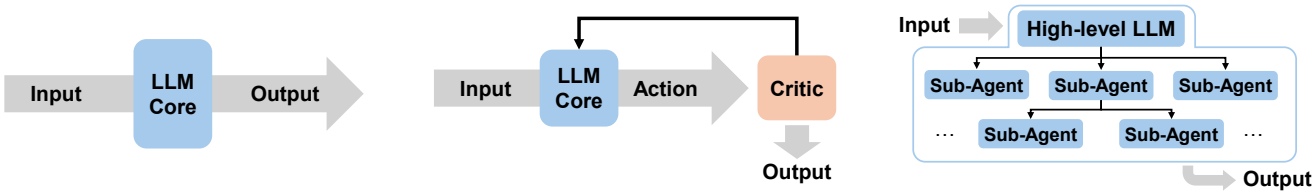
LLM Agents are fundamentally defined by their ability to extend the linguistic intelligence of the LLM core into autonomous action within an environment. This autonomy is enabled through the integration of modular components and structured control flow paradigms. In this section, we analyze the basic building blocks that constitute an Agent and the mechanisms that dictate the sequence of its actions.

A. Core Design Elements

The functional diversity of LLM Agents stems from the integration of three primary, non-exclusive modules around the central LLM:

The LLM Core: The Brain The Large Language Model itself serves as the obligatory and central component—the ”Brain.” It handles complex reasoning, goal interpretation, plan generation, and translating internal thoughts into executable actions or tool inputs. The quality of the underlying LLM (e.g., its context window size and reasoning capabilities) significantly impacts the Agent’s overall performance.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:37:27 UTC from IEEE Xplore. Restrictions apply.



(a) Single-Pass Execution (b) Iterative Self-Improvement Loop (c) Hierarchical Control

Fig. 2: Control flow paradigms in LLM agents

Memory Module The memory module provides the Agent with the ability to maintain state and learn over time. This is critical for tasks requiring consistency or long-term engagement.

- Short-Term Memory (STM): Managed via the LLM's context window, storing immediate conversation history, current planning steps, and tool outputs. STM is essential for coherence within a single task session [1], [6], [7].
- Long-Term Memory (LTM): Typically implemented using a Retrieval-Augmented Generation (RAG) architecture, where past experiences, general knowledge, or external documents are stored in a vector database and retrieved based on relevance to the current task [5], [8]–[10].

Planning Module This module enables the agent to decompose complex goals into manageable sub-tasks and formulate multi-step strategies [1], [11], [12]. Planning approaches can be categorized into three types: single-path planning generates a complete plan upfront before execution, adaptive planning dynamically adjusts plans based on intermediate results, and hierarchical planning breaks down high-level goals into nested sub-goals. The ReAct framework [1] exemplifies adaptive planning through its iterative Thought-Action-Observation cycle, where each observation informs the next planning step.

Tool/Action Module This module allows the Agent to interact with the external world beyond its training data. The ability to use tools is essential for grounding facts, executing complex computations, and taking physical action.

- Tool Selection and Input: The LLM generates the arguments required to call external functions, APIs, or specialized modules [2], [13], [14].
- Execution and Observation: The tool executes the action (e.g., searching the web, running Python code) and returns an observation [15], which is then fed back into the LLM Core for subsequent decision-making, exemplified by the Observation step in ReAct [1].

Beyond these core modules, agents may incorporate additional specialized components depending on their application. For instance, a reflection or critic module enables self-evaluation and iterative refinement of outputs [6], [8], while a profile module can store agent-specific characteristics for social simulation tasks [3]. However, memory, planning, and tool use remain the fundamental building blocks present in most agent architectures.

B. Control Flow Paradigms

Control flow refers to the mechanisms that guide the sequence of the Agent's thought, action, and planning processes. We identify three key paradigms:

Single-Pass Execution (Figure 2a) This is the simplest form, where the Agent follows a sequence of pre-defined or dynamically generated steps to reach a conclusion without an explicit self-correction loop. Examples include simple ReAct [1] chains (Thought-Action-Observation) where the sequence terminates upon task completion or error.

Iterative Self-Improvement Loop (Figure 2b) This paradigm introduces an explicit Reflection or Critic module to evaluate the outcome of an action or the current state of the plan [8], [16], [17]. If the outcome is unsatisfactory, the Agent revises its plan or previous actions and re-enters the loop. This mechanism, central to works like Reflexion [8], allows for robust error handling and complex goal attainment over multiple steps.

Hierarchical Control (Figure 2c) Hierarchical Agents structure the problem-solving process by delegating tasks [5], [18], [19]. A high-level LLM formulates a coarse-grained plan or goal, which is then decomposed into sub-tasks and managed by lower-level, specialized modules or other Agents. This is highly effective for long-horizon, open-ended tasks, as demonstrated by Voyager in continuous environment exploration [5].

III. Application Domains

The modular nature of LLM Agents allows for adaptation across diverse application domains. This section classifies these applications based on their primary goals.

A. Knowledge and Data Management

Agents in this domain focus on extending the knowledge boundary of the LLM and automating data analysis tasks [20], [21]. They heavily leverage the Memory Module and Tool/Action Module.

- Data Analysis and Reasoning: Agents translate natural language queries (e.g., "Analyze Q3 sales data") into executable code (e.g., Python, SQL) and interpret the results to provide structured insights. This is often realized through single-agent systems embodying the Iterative Self-Improvement Loop.
- Enhanced Retrieval and QA: Agents use sophisticated RAG techniques to dynamically search vast external

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:37:27 UTC from IEEE Xplore. Restrictions apply.

databases or the web, integrating real-time information to overcome the LLM's knowledge cutoff. Practical systems like Perplexity AI demonstrate this effective synthesis of search and reasoning.

B. Software and Engineering

This domain involves Agents automating the software development lifecycle (SDLC), from ideation to testing. They require robust Planning and complex Tool Use for code execution and environment manipulation [4], [22]–[24].

- Autonomous Code Generation: Agents can generate entire functions or applications based on high-level prompts. Systems like Auto-GPT [23] and AgentGPT [24] were early examples focusing on self-driven goal decomposition and iterative code refinement.
- Collaborative Development Teams: Multi-Agent systems are employed to mimic human engineering teams. MetaGPT [4] exemplifies this by assigning specialized roles (Product Manager, Engineer, Tester) to individual Agents that communicate and collaborate to produce final products.

C. Simulation and Interaction

Agents here are designed to mimic human or complex entity behavior within virtual, dynamic environments, primarily focusing on long-term consistency and social interaction [25], [26].

- Social Simulation: Agents are endowed with detailed profiles, memories, and reflection capabilities to generate believable and complex social behaviors. Generative Agents [3] demonstrated a large-scale simulation of human behavior, highlighting the importance of Memory and Reflection modules in maintaining social consistency.
- Embodied Control and Gaming: Agents are placed in interactive environments and must autonomously acquire skills and navigate the world. Voyager [5], utilizing Hierarchical control, learns to explore and acquire complex skills without human intervention.

IV. Challenges and Future Research Directions

Despite the promising capabilities demonstrated in Sections II and III, the field of LLM Agents is still nascent and faces several fundamental challenges that dictate future research directions.

A. Reliability and Non-Determinism

The most pressing challenge is the inherent reliability of Agent actions. Since the LLM Core is probabilistic, Agents often exhibit non-deterministic behavior, leading to unpredictable failure modes, such as:

- Hallucination in Planning: The agent generates a plausible but impossible or nonsensical plan.
- Action Misalignment: The agent selects the wrong tool or generates incorrect arguments for a valid tool.
- Repetitive Loops: Agents fail to recognize when they are stuck in a cycle of failed attempts and repeated actions, wasting computational resources.

Future research must focus on developing more robust and verifiable planning algorithms, potentially by integrating formal methods or structured verification layers into the control flow.

B. Evaluation and Benchmarking

Current LLM benchmarks (e.g., MMLU, GSM8K) are designed for static, single-turn tasks. Emerging agent benchmarks like ALFWorld [27] and WebShop [28] represent progress but remain limited to specific domains (embodied instruction following, web navigation) and short-horizon scenarios. To comprehensively evaluate agent systems, future research must develop:

- Extended-Duration Evaluation Frameworks: Frameworks assessing long-horizon planning (days to months), persistent memory management, and contextual consistency across extended interactions.
- Holistic Agent Performance Metrics: Evaluation criteria beyond task success, including planning quality, adaptive reasoning, tool-use efficiency, and collaboration effectiveness in Multi-Agent Systems.

C. Scalability and Efficiency

The current iterative and reflective nature of agents is computationally expensive. Running complex Agent systems, especially Multi-Agent Simulations, requires multiple LLM calls for a single decision cycle.

- Efficient Reflection: Minimizing the number of LLM calls required for effective self-correction.
- Knowledge Pruning: Developing memory management techniques that efficiently retrieve only the most pertinent information (LTM) without overwhelming the context window (STM).

D. Future Research Directions

Beyond addressing current limitations, the field is moving towards several exciting avenues:

- Multi-Modal Agents: Integrating visual, auditory, and tactile perception with the LLM Core, enabling Agents to interact with the physical world or complex digital interfaces more effectively.
- Human-Agent Collaboration: Designing Agents not for full autonomy, but for seamless collaboration with human users, where the Agent intelligently identifies when to take action and when to seek human input.
- Safety, Ethics, and Control: As Agents become more autonomous and capable of taking consequential actions (e.g., executing code, trading assets), robust safety layers and ethical guidelines are paramount to prevent unintended or harmful outcomes.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:37:27 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

The LLM agent paradigm represents a critical step in transitioning LLMs from passive text generators to autonomous, goal-directed systems capable of interacting with complex, real-world environments. This survey systematically analyzed the rapidly evolving field of LLM agents by focusing on their foundational structural mechanisms and diverse domain applications.

We first established in Section II that agent autonomy is achieved through the modular combination of the LLM core with three essential functional elements: the memory module, the planning module, and the tool/action module. The sophistication of an agent is determined by how these modules are orchestrated via structured control flow paradigms, such as the iterative self-improvement loop and hierarchical control.

Furthermore, we categorized the practical utility of these agents across key domains (Section III), demonstrating their impact from knowledge and data management to complex collaborative software engineering and dynamic simulation environments.

Despite this rapid progress, we concluded in Section IV that significant hurdles remain, particularly concerning the non-deterministic nature and reliability of agent actions and the absence of standardized long-horizon benchmarks for objective evaluation. Overcoming these challenges, alongside pursuing research in multi-modal agents and robust safety frameworks, will be crucial. The continued development of LLM agents signifies not just an incremental advance in AI technology, but a fundamental shift towards truly intelligent and autonomous computing entities.

Acknowledgment

This work was supported in part by the National Research Foundation of Korea (NRF) funded by the Korean government (MSIT) under Grant (RS-2025-02217071, RS-2025-00520248).

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *The eleventh international conference on learning representations*, 2022.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, pp. 68 539–68 551, 2023.

[3] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th annual acm symposium on user interface software and technology*, 2023, pp. 1–22.

[4] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2023.

[5] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[6] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong *et al.*, "Chatdev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15 174–15 186.

[7] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk *et al.*, "Graph of thoughts: Solving elaborate problems with large language models," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 16, 2024, pp. 17 682–17 690.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.

[10] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[11] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," *arXiv preprint arXiv:2305.04091*, 2023.

[12] E. H. Durfee, "Distributed problem solving and planning," in *ECCAI Advanced Course on Artificial Intelligence*. Springer, 2001, pp. 118–149.

[13] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan, "Gpt4tools: Teaching large language model to use tools via self-instruction," *Advances in Neural Information Processing Systems*, vol. 36, pp. 71 995–72 007, 2023.

[14] S. Wu, S. Zhao, Q. Huang, K. Huang, M. Yasunaga, K. Cao, V. Ioannidis, K. Subbian, J. Leskovec, and J. Y. Zou, "Avatar: Optimizing llm agents for tool usage via contrastive reasoning," *Advances in Neural Information Processing Systems*, vol. 37, pp. 25 981–26 010, 2024.

[15] B. Lei, Y. Zhang, S. Zuo, and C. Ding, "Autocoder: Enhancing code large language model with aiev-instruct," 2024.

[16] H. Sun, Y. Zhuang, L. Kong, B. Dai, and C. Zhang, "Adaplanner: Adaptive planning from feedback with language models," *Advances in neural information processing systems*, vol. 36, pp. 58 202–58 245, 2023.

[17] M. Jafaripour, S. Golestan, S. Miwa, Y. Mitsuka, and O. Zaiane, "Adaptive iterative feedback prompting for obstacle-aware path planning via llms," in *AAAI Workshop*, 2025.

[18] S. Qiao, N. Zhang, R. Fang, Y. Luo, W. Zhou, Y. E. Jiang, C. Lv, and H. Chen, "Autoact: Automatic agent learning from scratch for qa via self-planning," *arXiv preprint arXiv:2401.05268*, 2024.

[19] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570–578, 2023.

[20] Y. Lu and J. Wang, "Karma: Leveraging multi-agent llms for automated knowledge graph enrichment," *arXiv preprint arXiv:2502.06472*, 2025.

[21] S. Yue, S. Wang, W. Chen, X. Huang, and Z. Wei, "Synergistic multi-agent framework with trajectory learning for knowledge-intensive tasks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25 796–25 804.

[22] X. Guo, D. Keivan, U. Syed, L. Qin, H. Zhang, G. Dullerud, P. Seiler, and B. Hu, "Controlagent: Automating control system design via novel integration of llm agents and domain expertise," *arXiv preprint arXiv:2410.19811*, 2024.

[23] "AutoGPT," https://agpt.co/, Retrieved: October 23, 2025.

[24] "AgentGPT," https://agentgpt.reworkd.ai/, Retrieved: October 23, 2025.

[25] H. Li, C. Yang, A. Zhang, Y. Deng, X. Wang, and T.-S. Chua, "Hello again! llm-powered personalized agent for long-term dialogue," *arXiv preprint arXiv:2406.05925*, 2024.

[26] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "Llm experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2024, pp. 1–4.

[27] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, "Alfworld: Aligning text and embodied environments for interactive learning," *arXiv preprint arXiv:2010.03768*, 2020.

[28] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "Webshop: Towards scalable real-world web interaction with grounded language agents," *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 744–20 757, 2022.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:37:27 UTC from IEEE Xplore. Restrictions apply.